

Week 9 Final Submission

Production-Approaching Kubernetes Deployment

Student: Brain Kagai Macharia

Date: 2026-05-09

Course deliverable: single PDF per instructor template (manifests as verbatim text, required screenshots, 300–500 word production readiness assessment).

Contents

Part A — Kubernetes manifests copied verbatim from ~/k8s (not screenshots).

Part B — Required screenshots in the order listed on the assignment brief.

Part C — Written production readiness assessment.

Environment notes: Minikube with ingress-nginx addon; ingress reachability during development used kubectl port-forward service/ingress-nginx-controller 8080:80 with Host header kijani.local.

Screenshot checklist (Part B)

- `kubectl get all` showing both Deployments, all Pods Ready/Running, and both Services.
- `kubectl describe ingress` showing both rules with healthy Endpoints; `curl` output showing HTTP 200 for both `/payments` and `/api` routes.
- Mixed Pod state during Phase 5 stall (ImagePullBackOff alongside Running).
- `kubectl get pods` after rollback showing three Ready `kk-payments` pods on the restored ReplicaSet that matches the good `nginx:alpine` deployment.
- `kubectl rollout history` for `kk-payments` including deliberate regression and recovery revisions.

Part A — Manifest files (verbatim text from k8s/)

Files included: ingress, ConfigMaps, Deployments, Services, and kk-payments-secrets.yaml.example. Apply cluster Secrets using `kubectl create secret generic kk-payments-secrets ... -n kijani-project` as documented in the example — never commit live credentials.

Part A — kijani-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kijani-ingress
  namespace: kijani-project
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
  ingressClassName: nginx
  rules:
    - host: kijani.local
      http:
        paths:
          - path: /api(/|$)(.*)
            pathType: ImplementationSpecific
            backend:
              service:
                name: kk-api
                port:
                  number: 8080
          - path: /payments(/|$)(.*)
            pathType: ImplementationSpecific
            backend:
              service:
                name: kk-payments
                port:
                  number: 3000
```

Part A — kk-api-configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: kk-api-config
  namespace: kijani-project
  labels:
    app: kk-api
data:
  API_PORT: "8080"
  LOG_LEVEL: "info"
  NODE_ENV: "production"
  PAYMENTS_SERVICE_URL: "http://kk-payments:3000"
```

Part A — kk-api-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kk-api
  namespace: kijani-project
  labels:
    app: kk-api
  annotations:
    kubernetes.io/change-cause: "Initial production-approaching deployment: v1.0.0"
spec:
  replicas: 2
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: kk-api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: kk-api
    spec:
      containers:
        - name: kk-api
          image: nginx:alpine
          ports:
            - containerPort: 80
          envFrom:
            - configMapRef:
                name: kk-api-config
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 3
            periodSeconds: 10
            failureThreshold: 3
          livenessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 10
            periodSeconds: 20
            failureThreshold: 3
          resources:
            requests:
              cpu: "50m"
              memory: "64Mi"
            limits:
```

cpu: "250m"
memory: "128Mi"

Part A — kk-api-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: kk-api
  namespace: kijani-project
  labels:
    app: kk-api
spec:
  type: ClusterIP
  selector:
    app: kk-api
  ports:
    - name: http
      port: 8080
      targetPort: 80
      protocol: TCP
```


Part A — kk-payments-configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: kk-payments-config
  namespace: kijani-project
  labels:
    app: kk-payments
data:
  NODE_ENV: "production"
  DB_HOST: "postgres.kijani-project.svc.cluster.local"
  DB_PORT: "5432"
  LOG_LEVEL: "info"
  APP_PORT: "3000"
  MAX_CONNECTIONS: "20"
```

Part A — kk-payments-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kk-payments
  namespace: kijani-project
  labels:
    app: kk-payments
  annotations:
    kubernetes.io/change-cause: "Initial production-approaching deployment: v1.1.0"
spec:
  replicas: 3
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: kk-payments
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: kk-payments
    spec:
      containers:
        - name: kk-payments
          image: nginx:alpine
          ports:
            - containerPort: 3000
          envFrom:
            - configMapRef:
                name: kk-payments-config
            - secretRef:
                name: kk-payments-secrets
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 5
            periodSeconds: 10
            failureThreshold: 3
          livenessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 15
            periodSeconds: 20
            failureThreshold: 3
      resources:
        requests:
          cpu: "100m"
```

```
memory: "128Mi"  
limits:  
  cpu: "500m"  
  memory: "256Mi"
```

Part A — kk-payments-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: kk-payments
  namespace: kijani-project
  labels:
    app: kk-payments
spec:
  type: ClusterIP
  selector:
    app: kk-payments
  ports:
    - name: http
      port: 3000
      targetPort: 80
      protocol: TCP
```

Part A — kk-payments-secrets.yaml.example

```
# DOCUMENTATION ONLY – do not commit with real values
# To create this secret in your cluster run:
#
#   kubectl create secret generic kk-payments-secrets \
#     --from-literal=DB_PASSWORD=<your-db-password> \
#     --from-literal=STRIPE_API_KEY=<your-stripe-key> \
#     --from-literal=JWT_SECRET=<your-jwt-secret> \
#     -n kijani-project
#
# Required keys:
#   DB_PASSWORD      - PostgreSQL database password
#   STRIPE_API_KEY   - Stripe secret key (sk_live_... or sk_test_...)
#   JWT_SECRET       - Secret for signing JSON Web Tokens (min 64 chars)
```

Part B — Required screenshots

Evidence 1 — `kubectl get all -n kijani-project`

Evidence 2 — Ingress describe (healthy endpoints) and curl verification for `/payments` and `/api`

Evidence 3 — Mixed Pod state during Phase 5 stall (ImagePullBackOff alongside Running)

Evidence 4 — Rollback complete: `rollout undo/status` and `kubectl get pods` (three `kk-payments` pods Ready on restored ReplicaSet matching `v1.1.0` deployment)

Evidence 5 — `kubectl rollout history` for `kk-payments` (all revisions including rollback)

~ via ☕

> kubectl get all -n kijani-project

NAME	READY	STATUS	RESTARTS	AGE
pod/kk-api-85cd5ff494-gkd68	1/1	Running	0	29m
pod/kk-api-85cd5ff494-tn5g5	1/1	Running	0	29m
pod/kk-payments-7d7bc67584-mwl95	1/1	Running	0	3m14s
pod/kk-payments-7d7bc67584-vbds9	1/1	Running	0	32m
pod/kk-payments-7d7bc67584-zpswr	1/1	Running	0	32m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kk-api	ClusterIP	10.111.71.6	<none>	8080/TCP	29m
service/kk-payments	ClusterIP	10.97.201.247	<none>	3000/TCP	32m

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/kk-api	2/2	2	2	29m
deployment.apps/kk-payments	3/3	3	3	32m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/kk-api-85cd5ff494	2	2	2	29m
replicaset.apps/kk-payments-697c696b76	0	0	0	9m51s
replicaset.apps/kk-payments-7d7bc67584	3	3	3	32m

~ via ☕

> █

```
> curl -s -o /dev/null -w "%{http_code}" -H "Host: kijani.local" http://localhost:8080/payments/
200%
~ via ☕
> curl -s -o /dev/null -w "%{http_code}" -H "Host: kijani.local" http://localhost:8080/api/
200%
~ via ☕
> kubectl describe ingress kijani-ingress -n kijani-project
Name:          kijani-ingress
Labels:        <none>
Namespace:     kijani-project
Address:       192.168.49.2
Ingress Class: nginx
Default backend: <default>
Rules:
  Host          Path    Backends
  ----          -
  kijani.local  /api(/|$)(.*)    kk-api:8080 (10.244.0.28:80,10.244.0.27:80)
                /payments(/|$)(.*)  kk-payments:3000 (10.244.0.24:80,10.244.0.25:80,10.244.0.26:80)
Annotations:   nginx.ingress.kubernetes.io/rewrite-target: /$2
Events:
  Type    Reason    Age          From                      Message
  ----    -
  Normal  Sync      11m (x2 over 12m)  nginx-ingress-controller  Scheduled for sync

~ via ☕
> curl -v -H "Host: kijani.local" http://localhost:8080/payments/ 2>&1 | grep -E "< HTTP|Server|Connected"
* Connected to localhost (:::1) port 8080
< HTTP/1.1 200 OK

~ via ☕
> curl -v -H "Host: kijani.local" http://localhost:8080/api/ 2>&1 | grep -E "< HTTP|Server|Connected"
* Connected to localhost (:::1) port 8080
< HTTP/1.1 200 OK

~ via ☕
>
```



```
~ via ☕
> curl -v -H "Host: kijani.local" http://localhost:8080/payments/ 2>&1 | grep -E "< HTTP|Server|Connected"
* Connected to localhost (:::1) port 8080
< HTTP/1.1 200 OK

~ via ☕
[] curl -v -H "Host: kijani.local" http://localhost:8080/api/ 2>&1 | grep -E "< HTTP|Server|Connected"
* Connected to localhost (:::1) port 8080
< HTTP/1.1 200 OK

~ via ☕
> >....

ports:
  - containerPort: 3000
envFrom:
  - configMapRef:
      name: kk-payments-config
  - secretRef:
      name: kk-payments-secrets
readinessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 5
  periodSeconds: 10
  failureThreshold: 3
livenessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 15
  periodSeconds: 20
  failureThreshold: 3
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "256Mi"

EOF
```

```
[
~ via ☕
> kubectl apply -f k8s/kk-payments-deployment.yaml -n kijani-project
deployment.apps/kk-payments configured
```

```
~ via ☕
> kubectl get pods -n kijani-project -l app=kk-payments -w

NAME                                READY   STATUS             RESTARTS   AGE
kk-payments-697c696b76-4wd62       0/1     ErrImagePull       0           11s
kk-payments-697c696b76-87pbb       0/1     ErrImagePull       0           11s
kk-payments-7d7bc67584-vbds9       1/1     Running            0           22m
kk-payments-7d7bc67584-zpswr       1/1     Running            0           22m
kk-payments-697c696b76-4wd62       0/1     ImagePullBackOff   0           17s
kk-payments-697c696b76-87pbb       0/1     ImagePullBackOff   0           17s
kk-payments-697c696b76-4wd62       0/1     ErrImagePull       0           35s
kk-payments-697c696b76-87pbb       0/1     ErrImagePull       0           36s
kk-payments-697c696b76-4wd62       0/1     ImagePullBackOff   0           48s
kk-payments-697c696b76-87pbb       0/1     ImagePullBackOff   0           49s
kk-payments-697c696b76-87pbb       0/1     ErrImagePull       0           64s
kk-payments-697c696b76-4wd62       0/1     ErrImagePull       0           65s
kk-payments-697c696b76-87pbb       0/1     ImagePullBackOff   0           75s
kk-payments-697c696b76-4wd62       0/1     ImagePullBackOff   0           80s
kk-payments-697c696b76-4wd62       0/1     ErrImagePull       0          109s
kk-payments-697c696b76-87pbb       0/1     ErrImagePull       0          115s
kk-payments-697c696b76-4wd62       0/1     ImagePullBackOff   0           2m4s
kk-payments-697c696b76-87pbb       0/1     ImagePullBackOff   0           2m9s
```

```
~ via ☕
[> curl -s -o /dev/null -w "%{http_code}" -H "Host: kijani.local" http://localhost:8080/payments/
200%
~ via ☕
> kubectl rollout undo deployment/kk-payments --to-revision=1 -n kijani-project
Warning: resource deployments/kk-payments was previously managed with 'kubectl apply'. Rolling back will not update the kubectl.kubernetes.io/last-applied-configuration annotation, which may cause unexpected behavior on future 'kubectl apply' operations. Consider using 'kubectl apply' with your previous configuration file instead.
deployment.apps/kk-payments rolled back
[
~ via ☕
> kubectl rollout status deployment/kk-payments -n kijani-project
deployment "kk-payments" successfully rolled out

~ via ☕
> kubectl get pods -n kijani-project -l app=kk-payments
NAME                                READY   STATUS    RESTARTS   AGE
kk-payments-7d7bc67584-mwl95        1/1     Running   0           23s
kk-payments-7d7bc67584-vbds9        1/1     Running   0           29m
kk-payments-7d7bc67584-zpswr        1/1     Running   0           29m
[
~ via ☕
> █
```

~ via ☕

> kubectl rollout history deployment/kk-payments -n kijani-project

deployment.apps/kk-payments

REVISION	CHANGE-CAUSE
----------	--------------

2	Deliberate regression: v9.9.9-bad does not exist
---	--

3	Initial production-approaching deployment: v1.1.0
---	---

~ via ☕

> █

Part C — Production readiness assessment

This submission documents a production-approaching Kubernetes deployment for two cooperating services, `kk-api` and `kk-payments`, running in the dedicated namespace `kijani-project` on Minikube. The architecture separates concerns deliberately: non-sensitive configuration is delivered through ConfigMaps and consumed with `envFrom` so container images stay portable, while credentials for the payments tier are created imperatively with `kubectl` and only documented through an example manifest, avoiding accidental secret leakage into version control. Cluster networking exposes both workloads through a single `nginx` Ingress with host-based routing and rewrite rules so external clients see stable `/api` and `/payments` URLs while pods continue to serve traffic on simple HTTP roots suitable for the `nginx` stand-ins used here.

Operational safety is treated as a first-class requirement rather than an afterthought. Each Deployment declares explicit CPU and memory requests with matching limits so the scheduler can place pods predictably and noisy neighbours cannot starve critical tiers. Readiness probes gate traffic until pods can serve requests, while liveness probes restart containers that enter broken steady-state behaviour without confusing cold start with failure. RollingUpdate strategies differ intentionally: `kk-api` keeps `maxUnavailable` at zero to preserve headroom during changes, whereas `kk-payments` accepts bounded disruption across three replicas while still demonstrating that user-visible routes remained healthy during a deliberate bad-image rollout.

The Phase 5 exercise proves recoverability end to end. Introducing a non-existent image tag created authentic `ErrImagePull` and `ImagePullBackOff` behaviour on new `ReplicaSet` pods while older pods continued Running, which shows how Kubernetes surfaces partial failure without instantly draining capacity. Ingress-level checks returning HTTP 200 during that stall highlight why rollout discipline matters for customer-visible uptime. `kubectl rollout undo` returned the deployment to the known-good revision and `kubectl rollout history` captures an audit-friendly narrative via change-cause annotations, linking human intent to cluster state. The post-rollback `kubectl get all` snapshot confirms both Deployments ready, services wired as ClusterIPs, and the failed `ReplicaSet` scaled to zero.

Taken together, these choices illustrate the mindset Nia described: build manifests you can explain, roll forward safely, prove behaviour with commands you could run in an incident, and keep secrets out of Git. The embedded manifests reproduce the cluster definition exactly as applied, and the screenshots map directly to each required verification step for grading clarity.